Announcements

Topic 5: Review

By: Professor Lynam

Step Counting

```
int size = array.length;
for (int i = 0; i < size - 1; i++):
    for (int j = 0; j < size - i - 1; j++):
        if (array[j] > array[j + 1]):
            int temp = array[j]
            array[j] = array[j + 1]
            array[j + 1] = temp
```

- Outside for loops: 3
- Outer for loop: 4
- Inner for loop: 13
- Now… How to combine the for loops?

```
int size = array.length, o++ # variable assignment
o++ # outer for loop initialization
for (int i = 0; i < size - 1; i++):
        o++ # outer for loop true conditional evaluation
        o++ # inner for loop initialization
        for (int j = 0; j < size - i - 1; j++):
                o++ # inner for loop true conditional evaluation
                if (array[j] > array[j + 1]): (o++ * 6)
                        int temp = array[j] (o++)
                        array[j] = array[j + 1] (o++ * 2)
                        array[j + 1] = temp (o++ * 2)
                o++ # inner for loop incremenation
        o++; # inner for loop false conditional evaluation
        o++; # outer loop incrementation
o++; # outer for loop false conditional evaluation
```

Step Counting

```
int size = array.length;
for (int i = 0; i < size - 1; i++):
    for (int j = 0; j < size - i - 1; j++):
        if (array[j] > array[j + 1]):
            int temp = array[j]
            array[j] = array[j + 1]
            array[j + 1] = temp
```

- Outside for loops: 3, Outer for loop: 4, Inner for loop: 13
- The inner for loop goes from 0 to size (n) – i – 2 (n – i – 1 total iterations) where i starts at 0 and goes to n – 2 (n – 1 total iterations).
- We have to separate out the outer loop's iteration from the inner loop, using a summation for the inner loop. How many times will the inner loop run?
- $\sum_{i=0}^{n-2} n - i - 1$ is the summation for the inner loop. This is the total number of times it will run, so we just add it to the outer loop operations instead of multiplying it

Step Counting

```
int size = array.length;
for (int i = 0; i < size - 1; i++):
    for (int j = 0; j < size - i - 1; j++):
        if (array[j] > array[j + 1]):
            int temp = array[j]
            array[j] = array[j + 1]
            array[j + 1] = temp
```

- Outside for loops: 3, Outer for loop: 4, Inner for loop: 13, total number of inner loop iterations: $\sum_{i=0}^{n-2} n - i - 1$, total number of outer loop iterations: 4(n – 1).
- Combining it all together, we get: 3 + 4(n – 1) + $13 * \sum_{i=0}^{n-2} n - i - 1$.

Step Counting Practice

```
sum ← 0
while n ≥ 1:
        for j ← 1 through 8n:
                sum ← sum + 1
        n ← n / 2
```

- Nope! We still have the n back into the n -1, then multiply by 8!
- sum will equal 16n – 8. And the big-theta will be Θ(n)

- Find the big-Theta, and the value of sum after running the code.
- What values will n take for the inner for loop?
- n, n/2, n/4, …, 1
- When will $\frac{n}{2^i} = 1$?
- When $i = \log_2 n$. So our summation is $\sum_{i=1}^{\log_2 n} \frac{n}{2^i} = ???$
- $\sum_{i=1}^{\log_2 n} \frac{n}{2^i} = n * \left(1 - \frac{1}{n}\right) = n - 1$
- So the n/2 + … + 1 terms equal n – 1… are we done?

Step Counting Practice

```
int a = array_1.length;
int b = array_2.length;
int c = array_3.length;
int sum = 0;
for (int i = 0; i < a; i++):
        for (int j = 0; j < b; j++):
                for (int z = 0; z < c – 1; z++):
                        sum = array_1[i] + array_2[j] + array_3[z];
```

- Give this one a shot!
- Outside the for loops: 6
- First for loop: 4
- Second for loop: 4
- Third for loop: 11
- First for loop goes from 0 to a – 1 (a iterations), second from 0 to b – 1 (b iterations), third from 0 to c -2 (c – 1 iterations).
- Answer:
- 6 + a * (b * (11 * (c – 1) + 4) + 4)

Recurrence Relations

- Reminder of the process:

1. Determine the work required for the base and general cases of the given recursive algorithm.
2. Generate a few more equivalent recurrence for the work required in the general case.
3. Find the pattern within those expressions.
4. Create an equivalent closed-form (non-recurrence) expression in terms of the instance characteristic(s).
5. Prove that your closed-form expression is correct.
6. Determine the order of the algorithm.

Recurrence Relations

- Find the closed-form equivalent expression for the recurrence relation R(n) = R(n – 2) + k, where R(0) = c and n is an even integer. You do not need to provide a proof that your answer is correct.
- Which steps were you asked to perform for this question?
- 2-4. So let's start with 2! Generate 2 more equivalent recurrences.
- R(n) = R(n-4) + 2k
- R(n) = R(n-6) + 3k. Now step 3. What's the general form of this?
- R(n) = R(n – 2 * a) + a * k. Now step 4. How do we find what "a" is in terms of n?
- Since R(0) = c, let's find where n – 2 * a = 0. a = n/2, or n = 2 * a.
- Substitute that into our general expression: R(n) = R(0) + n/2*k, or R(n) = k * n / 2 + c

Recurrence Relations

- Prove that the closed-form of $T(n) = 2 * T\left(\frac{n}{2}\right) + n,$ where T(1) = c and n is a power of 2, is $T(n) = c * n + \text{ n} * \log_2 n.$
- Which step in the process is this?
- Step 5! I've already given a proposed closed-form expression, so no point in Finding the Pattern (which *cannot prove* anything)
- We need to do an inductive proof to show the closed-form expression is correct. We'll go through it, step-by-step.